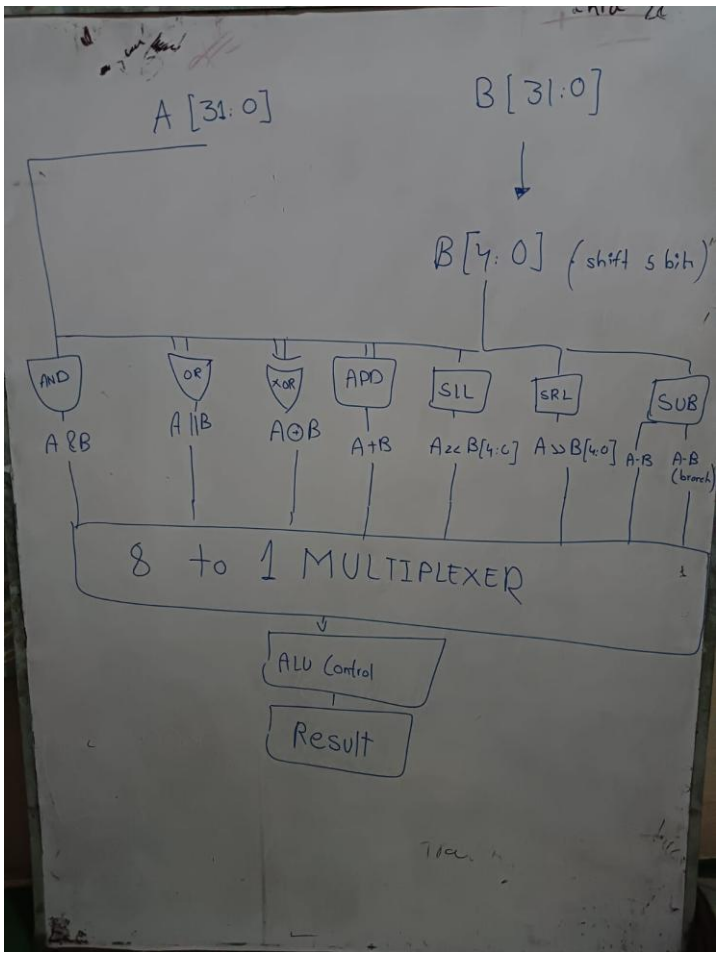
Lab 06 – Worksheet

Name: Mohammad Husain Naqvi	ID: 11062	Section: T-4

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Block Diagrams



Comments

Task 2

Modules + Testbench:

TOP Module:

```
21  module top_alu32bit_fpga (
22      input  wire        clk,
23      input  wire        btnC,
24      input  wire [15:0] sw,
25      output wire [15:0] led
26  );
27      wire rst_db;
28      debouncer db_rst (
29          .clk   (clk),
30          .pbin  (btnC),
31          .pbout (rst_db)
32      );
33      wire [31:0] sw_readData;
34      wire [15:0] sw_led_mirror;
35
36      switches sw_if (
37          .clk         (clk),
38          .rst         (rst_db),
39          .writeData   (32'b0),
40          .writeEnable (1'b0),
41          .readEnable  (1'b1),
42          .memAddress  (30'b0),
43          .sw          (sw),
44          .readData    (sw_readData),
45          .leds        (sw_led_mirror)
46      );
47      // Form 4-bit ALUControl: sw[2:0] mapped with MSB = 0
48      wire [3:0]  op = {1'b0, sw_readData[2:0]};
49      // Procedure Step (e): Fixed 32-bit operands
50      wire [31:0] A  = 32'h10101010;
51      wire [31:0] B  = 32'h01010101;
52
53      wire [31:0] result;
54      wire        zero;
55      // ALU instantiation with 4-bit control
56      ALU_32bit alu (
57          .A          (A),
58          .B          (B),
59          .ALUControl (op),
60          .Result     (result),
61          .Zero       (zero)
62      );
63      // Drive 16 board LEDs
64      assign led = result[15:0];
65  endmodule
66
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
//// Company:
//// Engineer:
////
//// Create Date: 08.10.2026 13:36:04
//// Design Name:
//// Module Name: ALUtop
//// Project Name:
//// Target Devices:
//// Tool Versions:
//// Description: A
////
//// Dependencies:
////
//// Revision:
//// Revision 0.01 - File Created
//// Additional Comments:
////
//////////////////////////////////////////////////////////////////////////////////
module top_alu32bit_fpga (
    input  wire        clk,
    input  wire        btnC,
    input  wire [15:0] sw,
    output wire [15:0] led
);
    wire rst_db;
    debouncer db_rst (
        .clk   (clk),
        .pbin  (btnC),
        .pbout (rst_db)
    );
    wire [31:0] sw_readData;
    wire [15:0] sw_led_mirror;

    switches sw_if (
        .clk         (clk),
        .rst         (rst_db),
        .writeData   (32'b0),
        .writeEnable (1'b0),
        .readEnable  (1'b1),
        .memAddress  (30'b0),
        .sw          (sw),
        .readData    (sw_readData),
        .leds        (sw_led_mirror)
    );
    // Form 4-bit ALUControl: sw[2:0] mapped with MSB = 0
    wire [3:0]  op = {1'b0, sw_readData[2:0]};
    // Procedure Step (e): Fixed 32-bit operands
    wire [31:0] A  = 32'h10101010;
    wire [31:0] B  = 32'h01010101;

    wire [31:0] result;
    wire        zero;
    // ALU instantiation with 4-bit control
    ALU_32bit alu (
```

```
      .A        (A),
      .B        (B),
      .ALUControl (op),
      .Result    (result),
      .Zero      (zero)
   );
   // Drive 16 board LEDs
   assign led = result[15:0];
endmodule
```

ALU :

```
module ALU_32bit (
    input  wire [31:0] A,
    input  wire [31:0] B,
    input  wire [3:0]  ALUControl,  // 4-b
    output reg  [31:0] Result,
    output wire        Zero
);

    always @(*) begin
        case (ALUControl)
            4'b0000: Result = A & B;
            4'b0001: Result = A | B;
            4'b0010: Result = A + B;
            4'b0011: Result = A ^ B;
            4'b0100: Result = A << B[4:0];
            4'b0101: Result = A >> B[4:0];
            4'b0110: Result = A - B;
            4'b0111: Result = A - B;
            default: Result = 32'b0;
        endcase
    end

    assign Zero = (Result == 32'b0);

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 08.10.2026 02:54:14
// Design Name:
// Module Name: acsd
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////

module ALU_32bit (
   input  wire [31:0] A,
   input  wire [31:0] B,
   input  wire [3:0]  ALUControl,  // 4-bit ALU control
   output reg  [31:0] Result,
   output wire        Zero
```

```
);

   always @(*) begin
      case (ALUControl)
         4'b0000: Result = A & B;       // AND (Switches: None) -> 0x0000
         4'b0001: Result = A | B;       // OR  (Switches: 0)    -> 0x1111
         4'b0010: Result = A + B;       // ADD (Switches: 1)    -> 0x1111
         4'b0011: Result = A ^ B;       // XOR (Switches: 0, 1) -> 0x1111
         4'b0100: Result = A << B[4:0];  // SLL (Switches: 2)    -> 0x2020
         4'b0101: Result = A >> B[4:0];  // SRL (Switches: 0, 2) -> 0x0808
         4'b0110: Result = A - B;       // SUB (Switches: 1, 2) -> 0x0F0F
         4'b0111: Result = A - B;       // BEQ (Switches: 0, 1, 2) -> 0x0F0F
         default: Result = 32'b0;
      endcase
   end

   assign Zero = (Result == 32'b0);

endmodule
```

Debouncer:

```
module debouncer (
    input  wire clk,
    input  wire pbin,
    output reg  pbout
);

    reg [19:0] counter = 20'b0;
    reg        sync_0  = 1'b0;
    reg        sync_1  = 1'b0;

    // Double-flop synchronizer
    always @(posedge clk) begin
        sync_0 <= pbin;
        sync_1 <= sync_0;
    end

    // Counter filter (~10ms at 100MHz)
    always @(posedge clk) begin
        if (sync_1 == pbout) begin
            counter <= 20'b0;
        end else begin
            counter <= counter + 1'b1;
            if (counter >= 20'd1_000_000) begin
                pbout   <= sync_1;
                counter <= 20'b0;
            end
        end
    end

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer: Muddassir Ali

// Module Name: debouncer
// Project Name: Counter
// Target Devices: Baasys 3
//
//////////////////////////////////////////////////////////////////////////////////

module debouncer (
   input  wire clk,
   input  wire pbin,
   output reg  pbout
```

```
);

    reg [19:0] counter = 20'b0;
    reg        sync_0  = 1'b0;
    reg        sync_1  = 1'b0;

    // Double-flop synchronizer
    always @(posedge clk) begin
        sync_0 <= pbin;
        sync_1 <= sync_0;
    end

    // Counter filter (~10ms at 100MHz)
    always @(posedge clk) begin
        if (sync_1 == pbout) begin
            counter <= 20'b0;
        end else begin
            counter <= counter + 1'b1;
            if (counter >= 20'd1_000_000) begin
                pbout   <= sync_1;
                counter <= 20'b0;
            end
        end
    end

endmodule
```

SWITHCES:

```
1  `timescale 1ns / 1ps
2
3
4  module switches (
5      input  wire        clk,
6      input  wire        rst,
7      input  wire [31:0] writeData,
8      input  wire        writeEnable,
9      input  wire        readEnable,
10     input  wire [29:0] memAddress,
11     input  wire [15:0] sw,
12     output reg  [31:0] readData,
13     output reg  [15:0] leds
14 );
15
16     always @(posedge clk or posedge rst) begin
17         if (rst) begin
18             readData <= 32'b0;
19             leds     <= 16'b0;
20         end else begin
21             if (readEnable) begin
22                 readData <= {16'b0, sw};
23             end
24             if (writeEnable) begin
25                 leds <= writeData[15:0];
26             end
27         end
28     end
29
30 endmodule
```

```
`timescale 1ns / 1ps

module switches (
   input  wire       clk,
   input  wire       rst,
   input  wire [31:0] writeData,
   input  wire       writeEnable,
   input  wire       readEnable,
   input  wire [29:0] memAddress,
   input  wire [15:0] sw,
   output reg  [31:0] readData,
   output reg  [15:0] leds
);

   always @(posedge clk or posedge rst) begin
      if (rst) begin
         readData <= 32'b0;
         leds     <= 16'b0;
      end else begin
         if (readEnable) begin
            readData <= {16'b0, sw};
         end
         if (writeEnable) begin
            leds <= writeData[15:0];
         end
      end
   end

endmodule
```

TESTBENCH:

```
`timescale 1ns / 1ps

module ALU_tb;

   reg  [31:0] A;
   reg  [31:0] B;
   reg  [3:0]  ALUControl;
   wire [31:0] Result;
   wire       Zero;

   ALU_32bit uut (
      .A         (A),
      .B         (B),
      .ALUControl (ALUControl),
```

```
    .Result     (Result),
    .Zero       (Zero)
);

initial begin
    // Fixed operands per step (e)
    A = 32'h10101010;
    B = 32'h01010101;
    ALUControl = 4'b0000;
    #50;

    // 1. AND (0000) -> 0x00000000
    ALUControl = 4'b0000;
    #50;

    // 2. OR (0001)  -> 0x11111111
    ALUControl = 4'b0001;
    #50;

    // 3. ADD (0010) -> 0x11111111
    ALUControl = 4'b0010;
    #50;

    // 4. XOR (0011) -> 0x11111111
    ALUControl = 4'b0011;
    #50;

    // 5. SLL (0100) -> 0x20202020
    ALUControl = 4'b0100;
    #50;

    // 6. SRL (0101) -> 0x08080808
    ALUControl = 4'b0101;
    #50;

    // 7. SUB (0110) -> 0x0F0F0F0F
    ALUControl = 4'b0110;
    #50;

    // 8. BEQ (0111) -> 0x0F0F0F0F
    ALUControl = 4'b0111;
    #50;

    // 9. Zero Flag Check (Equal inputs)
    A = 32'hA5A5A5A5;
    B = 32'hA5A5A5A5;
    ALUControl = 4'b0110; // SUB -> Result = 0, Zero = 1
    #50;
```

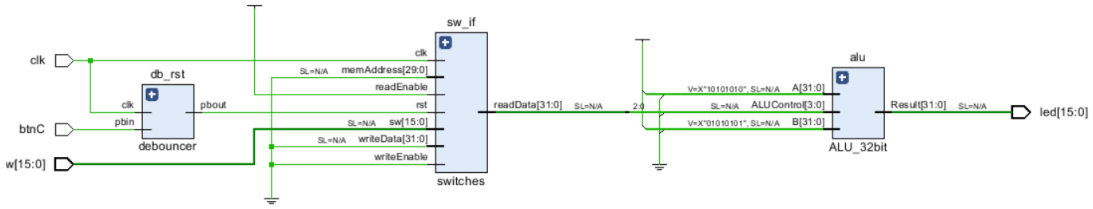
```
    $finish;
  end

endmodule
```

Results:

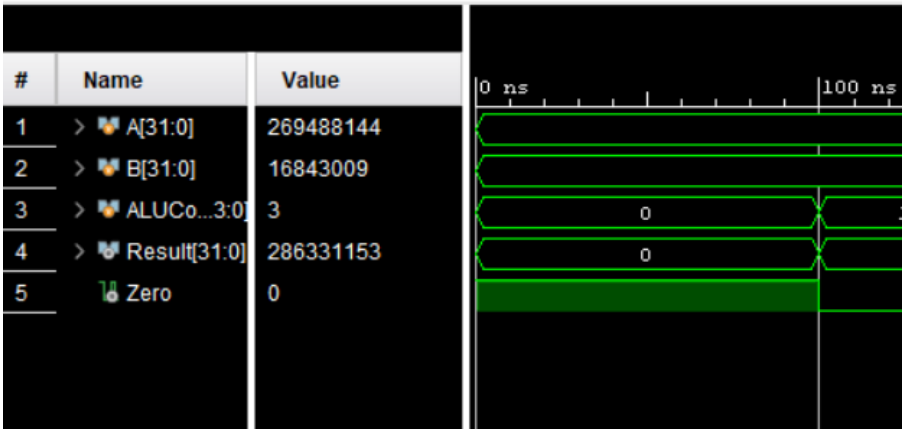
Task 3

RTL Schematics

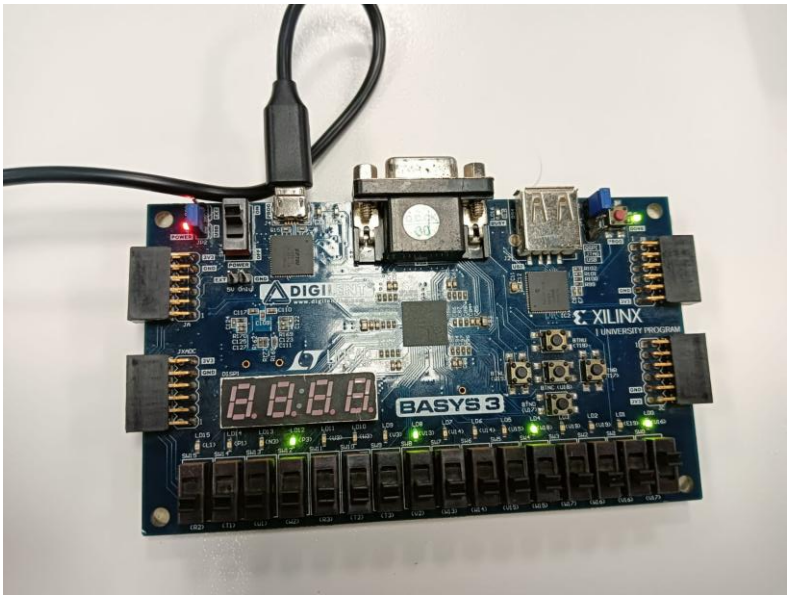


The simulation tests all ALU operations using fixed inputs
A = 0x10101010 (269488144)
B = 0x01010101 (16843009).

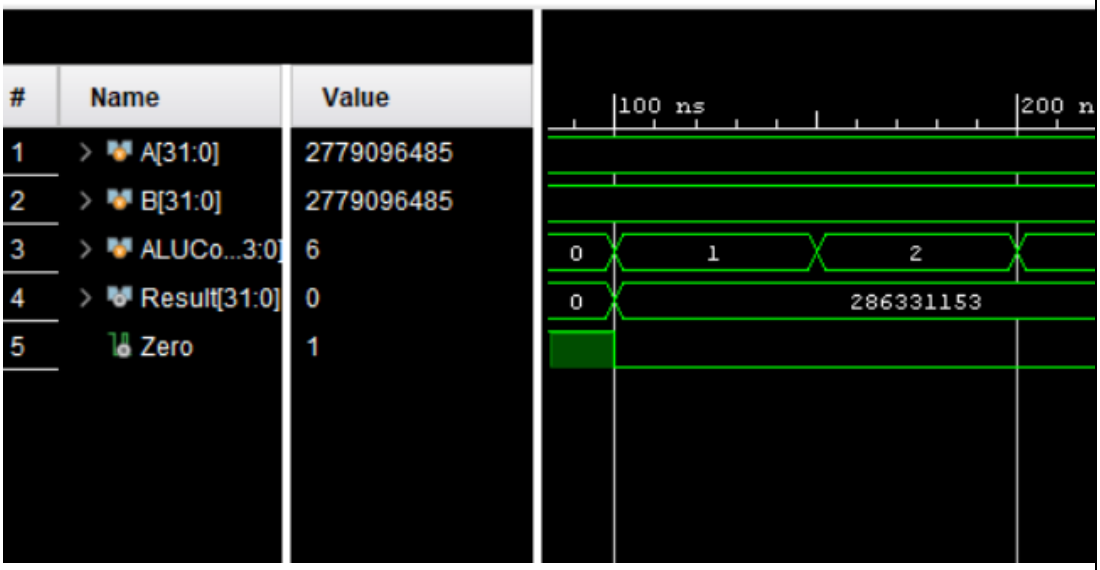
AND: (0 – 100ns)

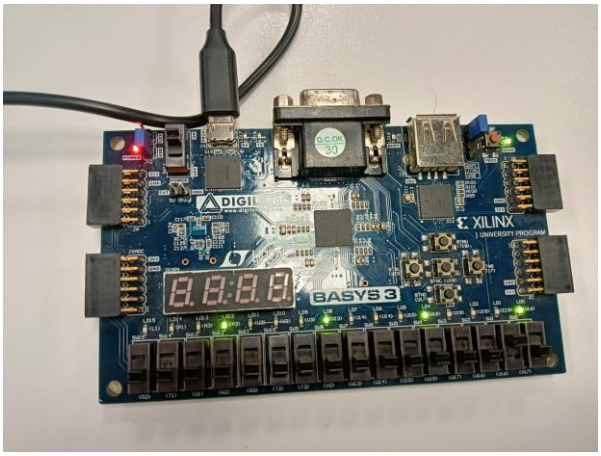


1. AND (ALUControl = 0): Result = 0 (0x00000000). All bits are 0, so Zero flag =



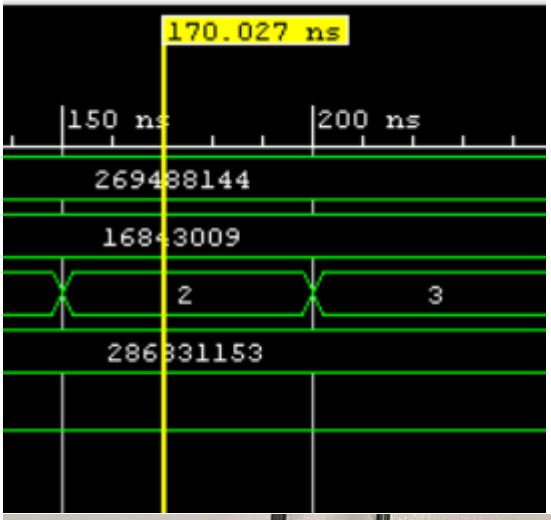
OR: (100ns – 170 ns)

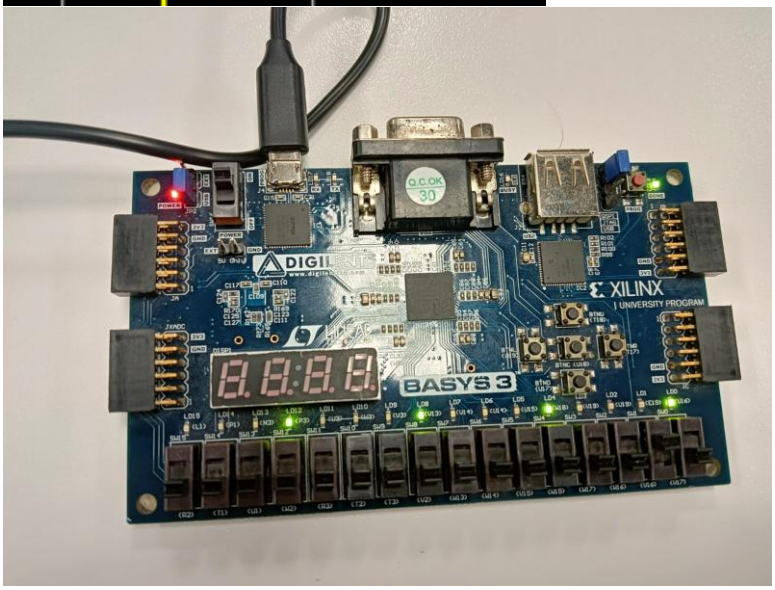




2. OR (ALUControl = 1): Result = 286331153 (0x11111111), Zero flag = 0.

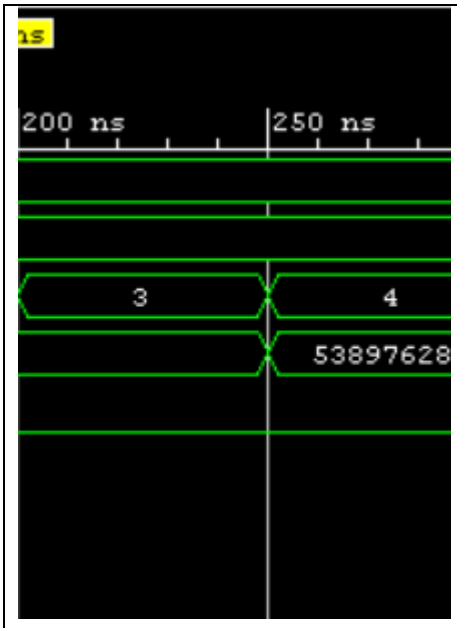
ADD: (170 -200ns)

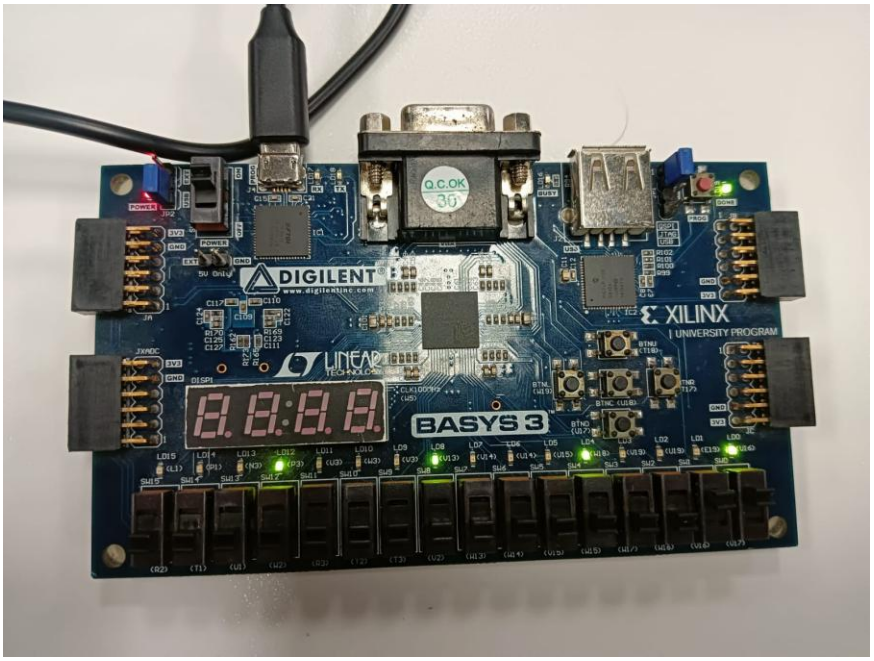




ADD (ALUControl = 2): Result = 286331153 (0x11111111), Zero flag = 0.

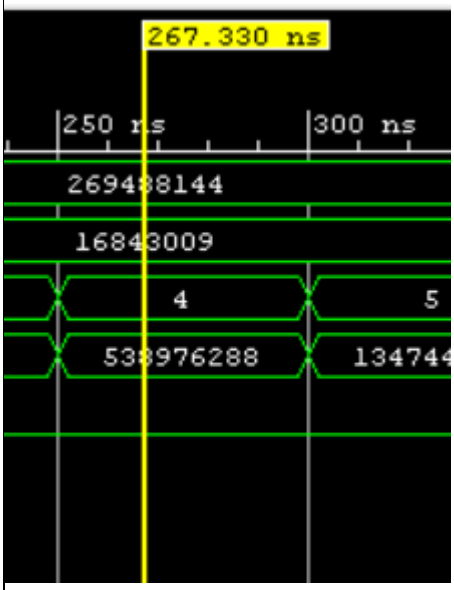
XOR: (200 – 270 ns)

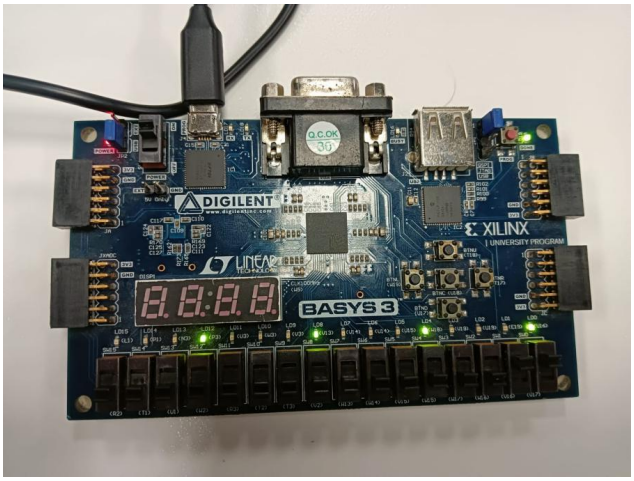




XOR (ALUControl = 3): Result = 286331153 (0x11111111), Zero flag = 0.

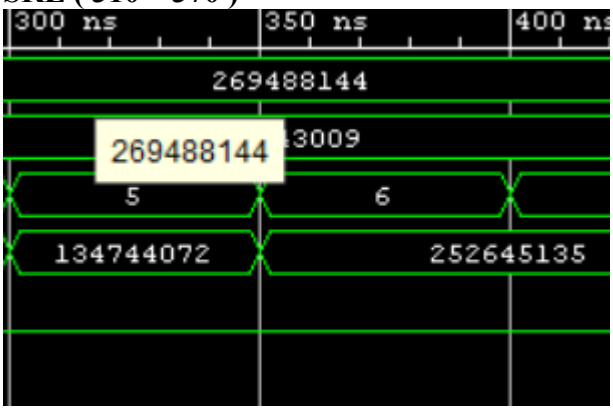
SLL: (270 – 310)ns

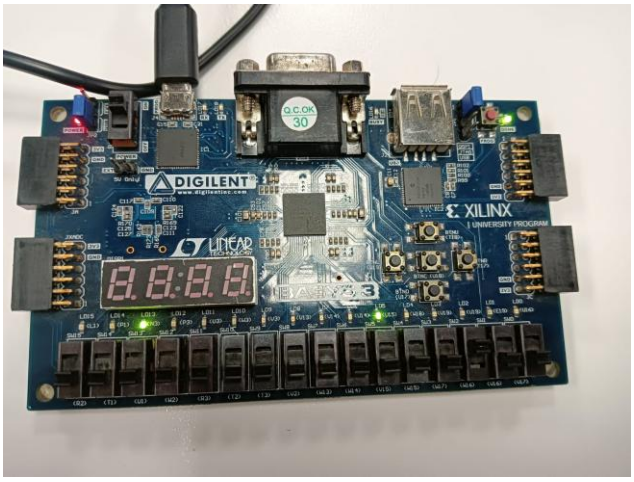




SLL (ALUControl = 4): Shifts A left by B[4:0] (1 bit). Result = 538976288 (0x20202020).

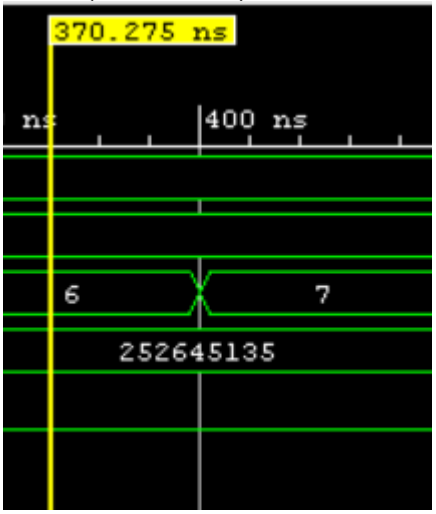
SRL (310 – 370)

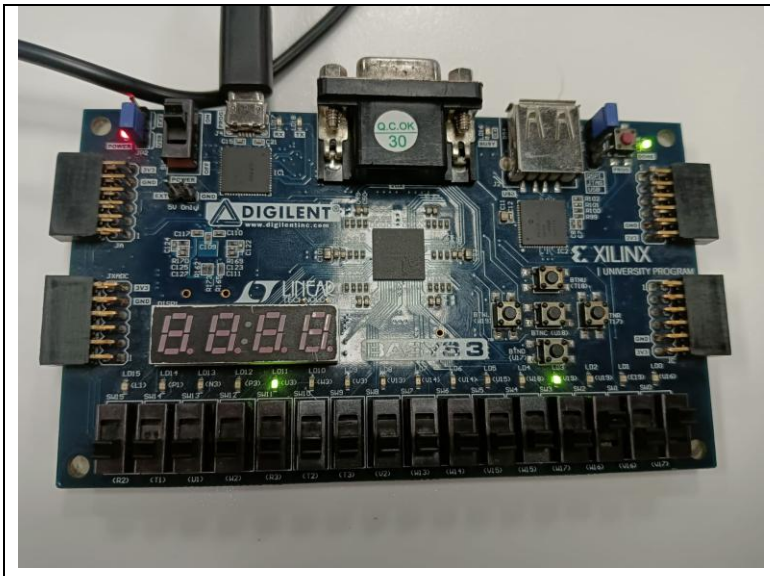




SRL (ALUControl = 5): Shifts A right by B[4:0] (1 bit). Result = 134744072 (0x08080808).

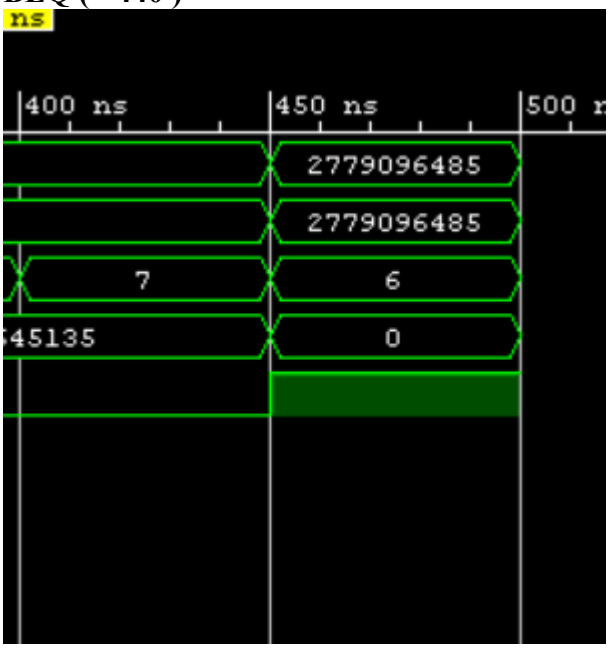
SUB: (370 – 420)ns

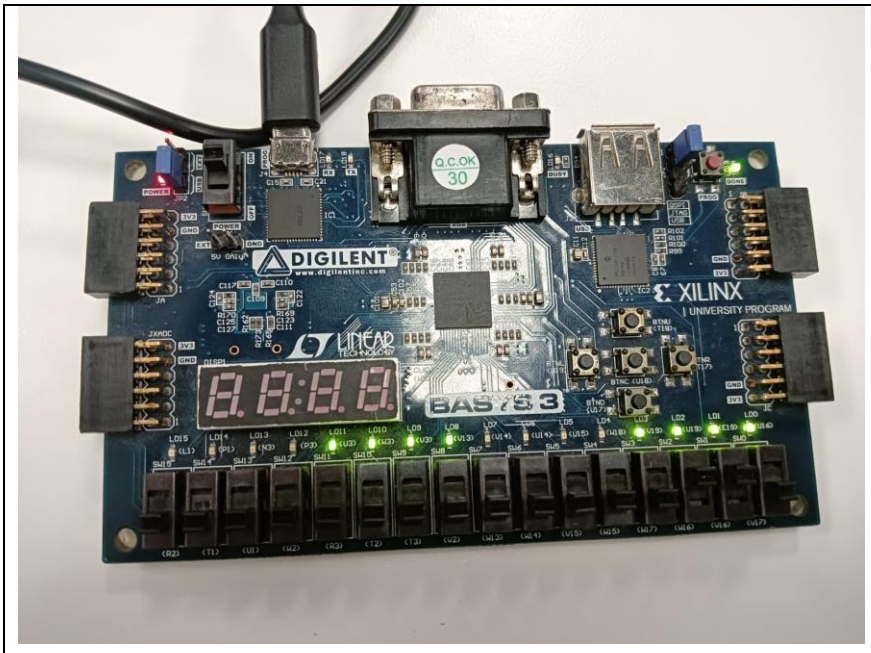




SUB (ALUControl = 6): A - B gives Result = 252645135 (0x0F0F0F0F).

BEQ (<440)





BEQ (ALUControl = 7): Executes subtraction datapath; Result = 252645135 (0x0F0F0F0F).

GithubLink:
https://github.com/MohammadNaqvi1098/CALabs-Git-mn11062.git

Lab 05 – Designing FSM Using FPGA Switches and LEDs
Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/20	/20
Task 2	/20	/15		
Task 3	/10	/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	**Program/Code / Simulation Model/ Network Model**	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	**Results & Plots**	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	**Report**	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.